INEEL
INSTITUTO NACIONAL
DE ELECTRICIDAD Y
ENERGÍAS LIMPIAS

Guía Estratégica de Auditoría de Sistemas Web: Implementación del Estándar OWASP Top 10:2025

Robbie Daniel Mejía Arroyo

Lunes, 27 de abril del 2026

Contenido

Introducción 3
A01:2025 – Control de Acceso Roto 4
A02:2025 – Configuración de Seguridad Incorrecta 7
A03:2025 – Fallas en Cadena de Suministro 11
A04:2025 – Fallas Criptográficas 14
A05:2025 – Inyección 17
A06:2025 – Diseño Inseguro 20
A07:2025 – Fallas en la Identificación y Autenticación 21
A08:2025 – Fallas en la Integridad de Software y Datos 23
A09:2025 – Fallas en el Registro y Monitoreo Control 26
A10:2025 – Mal Manejo de Condiciones Excepcionales 28
Conclusión de la Auditoría 30

Introducción

Esta guía tiene como objetivo estandarizar las pruebas de seguridad en los sistemas web de la institución (basados en arquitecturas modernas como .NET 8 y C#) para asegurar que cada desarrollo cumpla con los niveles de protección requeridos ante amenazas actuales.

Para fines prácticos y demostrativos, las secciones de Malas Prácticas Comunes y Ejecución de Pruebas, Recomendaciones Técnicas y los Checklists de esta guía se basan en el análisis de seguridad realizado en la plataforma Andrómeda tomando como marco de referencia el estándar internacional OWASP Top 10 del 2025, el cual identifica los riesgos de seguridad más críticos para las aplicaciones web modernas. Durante el periodo de enero - abril del 2026.

Andrómeda, representada por la Figura 1 Logotipo de la plataforma Andrómeda, es una herramienta institucional de Inteligencia de Negocios (BI) diseñada para el procesamiento de modelos económicos y datos del sector energético. A través de este documento, se explicará paso a paso cómo replicar las pruebas y evaluaciones que se ejecutaron en dicho sistema para que puedan ser aplicadas con éxito en cualquier otra plataforma de la institución.

Figura 1 Logotipo de la plataforma Andrómeda

Cómo iniciar una investigación de seguridad

Para realizar una auditoría desde cero, el auditor debe seguir estos pasos fundamentales:

1. Reconocimiento y Referencia: Consultar el sitio oficial de OWASP.org para identificar los riesgos críticos del año vigente.
2. Definición de Alcance: Identificar si el sistema es una API, una aplicación monolítica o microservicios para seleccionar las herramientas adecuadas.
3. Selección de Herramientas: Dependiendo del objetivo (red, código o tráfico), se deben elegir herramientas de escaneo (Nmap), manipulación de solicitudes (curl/Burp Suite) o análisis dinámico (OWASP ZAP).
4. Ejecución y Evaluación: Probar no solo que el sistema funcione, sino cómo reacciona ante entradas malintencionadas o accesos no autorizados

A01:2025 – Control de Acceso Roto

Descripción Detallada

El control de acceso garantiza que los usuarios no puedan actuar fuera de sus permisos establecidos. Esta vulnerabilidad ocurre cuando un sistema falla en restringir el acceso a recursos o funciones, permitiendo que un atacante vea, modifique o elimine datos a los que no debería tener acceso.

Ejemplos Específicos (General)

- Inseguridad en Identificadores (ID): Modificar un parámetro en la URL (ej. de ID=100 a ID=101) para ver perfiles de otros usuarios.
- Elevación de Privilegios: Un usuario con rol de "Consultor" que logra acceder a funciones de "Administrador" al conocer la ruta directa de una página.
- Falta de Autorización en Endpoints: APIs que no validan el token de acceso en cada petición, confiando únicamente en que el usuario ya inició sesión en el navegador.

Malas Prácticas Comunes y Ejecución de Pruebas en la plataforma de Andrómeda

Para identificar estas fallas en nuestros sistemas, se deben ejecutar algunas de las siguientes tácticas de descubrimiento:

A. Identificación de Recursos sin Autenticación

Dejar puertos o interfaces de desarrollo expuestas en producción sin capas de seguridad.

Herramienta: Nmap (Network Mapper).

Instalación: En sistemas Linux/macOS usar sudo apt install nmap. En Windows, descargar el instalador oficial (https://nmap.org/download).

Ejecución: Para descubrir servicios y sistemas operativos expuestos, se utiliza el comando:

```
sudo nmap -O [IP_SERVIDOR]
```

Sintaxis: sudo (permisos de administrador), nmap (ejecuta el programa), -O (activar detección de Sistema Operativo y versiones).

Resultado del análisis: Mediante este método se localizaron APIs activas con acceso abierto en el rango de puertos 5000 como se puede ver en la Figura 2 APIs expuestas.

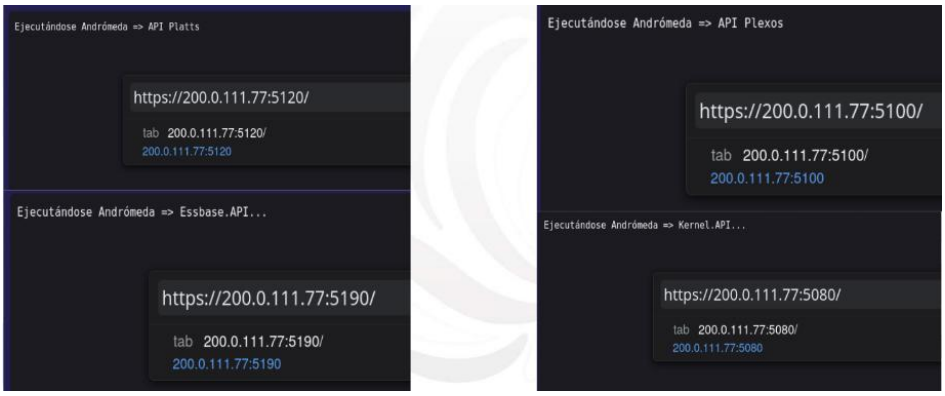


Figura 2 APIs expuestas

B. Validar Transferencia de Recursos Externos

Error común: Permitir que el servidor acepte peticiones de carga o descarga de archivos desde entornos ajenos a la plataforma.

Herramienta: curl.

Ejecución: Se utiliza para simular peticiones HTTP directas al servidor:

curl -I http://[IP_SERVIDOR]/recurso

Resultado del análisis: El sistema denegó correctamente los intentos de descarga de archivos en entornos externos.

C. Verificar la Segregación de Funciones (Roles)

Error común: Basar la seguridad solo en el menú visual (ocultar botones) pero no proteger la ruta lógica en el servidor (Acceso horizontal/vertical).

Método: Pruebas Manuales de Navegación Forzada.

Ejecución: Iniciar sesión con un perfil de menor jerarquía e intentar ingresar manualmente a las secciones exclusivas de un perfil con mayores privilegios.

Resultado del análisis: Los mecanismos de seguridad bloquearon los accesos cruzados entre perfiles, emitiendo alertas de sesión correspondientes mostrado como en la Figura 3 Accesos no autorizados.

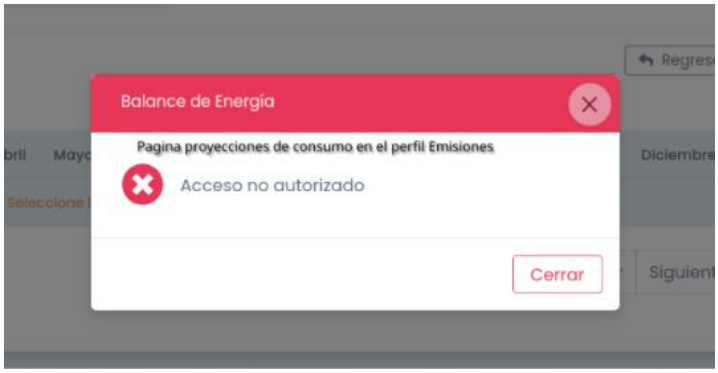


Figura 3 Accesos no autorizados

Recomendaciones Técnicas

- Seguridad en APIs: Implementar RBAC (Role-Based Access Control) y validar la autorización en cada microservicio o endpoint.
- Validación del Lado del Servidor: No confiar en las restricciones visuales; cada petición debe ser validada contra las políticas de acceso definidas.
- Restricción de APIs: Cerrar o aplicar seguridad estricta en todas las APIs, restringiendo el acceso solo a usuarios autenticados mediante tokens seguros.

Checklist para Auditoría Continua (Andrómeda 2026)

Punto de Control	**Estado de Cumplimiento**
Identificar recursos sin autenticación	[✗] No Cumple
Validar transferencia de archivos externos	[✓] Cumple
Verificar la segregación de funciones (Roles)	[✓] Cumple

A02:2025 – Configuración de Seguridad Incorrecta

Descripción Detallada

Este riesgo se refiere a la falta de endurecimiento (hardening) en cualquier nivel de la pila de la aplicación, incluyendo servicios de red, servidores web, bases de datos y la propia aplicación. Una configuración insegura puede permitir que un atacante obtenga información del sistema, explote servicios innecesarios o tome control del entorno.

Ejemplos Específicos (General)

- Servicios innecesarios activos: Puertos abiertos que no tienen una función crítica para el negocio.
- Cuentas predeterminadas: Mantener usuarios y contraseñas de fábrica en consolas de administración.
- Desvelación de errores técnicos: Mostrar detalles del código o versiones de software en mensajes de error públicos.

Malas Prácticas Comunes y Ejecución de Pruebas (Basado en el análisis de Andrómeda)

A. Auditoría de Cabeceras de Seguridad (CSP)

Error común: No definir una Política de Seguridad de Contenido (CSP), facilitando ataques de Inyección y XSS.

Herramienta: OWASP ZAP (Zed Attack Proxy) (https://www.zaproxy.org/download/).

Metodología Paso a Paso:

Configuración de Objetivo: Abrir OWASP ZAP e ingresar la URL del sistema en la sección "Quick Start".

Escaneo Automático: Presionar el botón "Attack". La herramienta realizará un rastreo (spider) y lanzará ataques pasivos/activos para evaluar las respuestas del servidor (ver Figura 4 Ataque automatizado con ZAP).

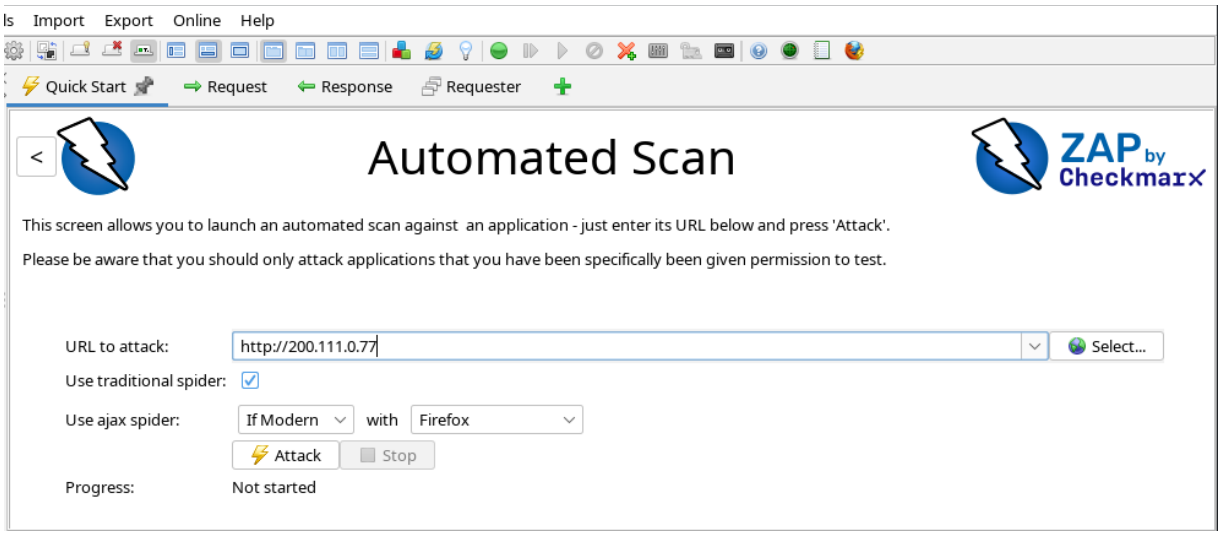


Figura 4 Ataque automatizado con ZAP

Análisis de Alertas: En la pestaña inferior de "Alerts", ZAP categoriza los hallazgos por severidad (Rojo: Alta, Naranja: Media, Amarillo: Baja).

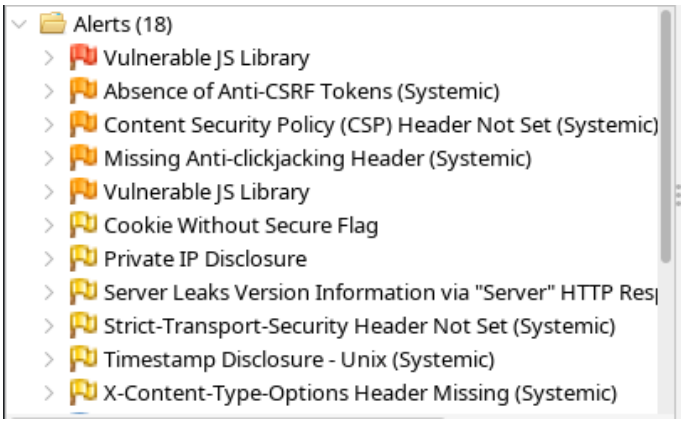


Figura 5 Alertas de vulnerabilidades detectadas con ZAP

Resultado del análisis: Se detectó la ausencia de la directiva CSP, categorizada como una alerta de severidad media-alta, lo que permitiría la ejecución de scripts externos no autorizados.

B. Evaluación de Exposición de Infraestructura

Herramienta: Nmap.

Ejecución: `sudo nmap -sV -Pn [IP_SERVIDOR]`

Sintaxis: -sV (determina la versión del servicio), -Pn (omite el descubrimiento de host si hay firewalls).

Resultado del análisis: Se identificaron puertos activos relacionados con motores SQL y el Puerto 445, el cual está históricamente asociado a vulnerabilidades de ejecución remota de código como WannaCry.

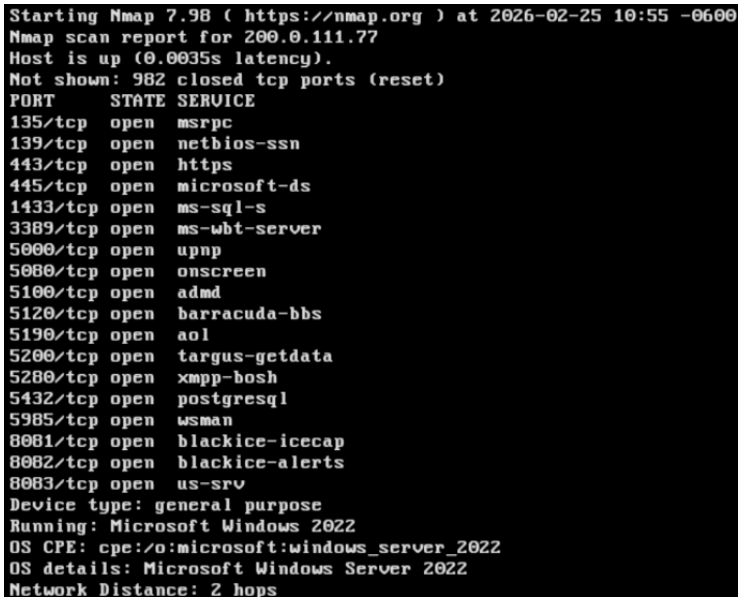


Figura 6 Evaluación de puertos con Nmap

C. Análisis de Mensajes de Error y Depuración

Método: Pruebas manuales de error.

Procedimiento: Ingresar datos aleatorios o malformados en los campos de entrada de la aplicación para forzar excepciones del sistema.

Resultado del análisis: El sistema arrojó mensajes de error por defecto que revelan el uso de .NET 8 / C#, facilitando al atacante el reconocimiento del entorno de ejecución.

D. Exposición de Stack Traces y APIs

Método: Auditoría de respuestas de API mediante pruebas de tráfico manual.

Procedimiento: Navegar por las diferentes secciones de la plataforma e introducir tipos de datos inválidos en los parámetros de las consultas.

Resultado del análisis: Se observó la presencia de trazas de error (stack traces) que exponen la estructura interna de las consultas en ciertos endpoints, lo cual es información crítica para un atacante.

E. Identificación del Sistema Operativo (OS)

Herramienta: OWASP ZAP (Análisis de tráfico).

Procedimiento: Durante el escaneo automático, se inspeccionan las cabeceras de respuesta HTTP (específicamente la cabecera Server).

Resultado del análisis: Se detectó que el nombre y versión del servidor web están expuestos directamente: IIS 10.0.

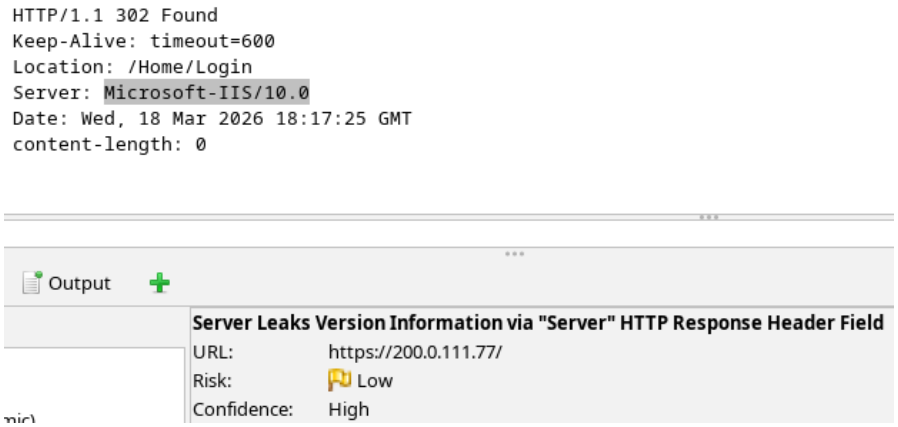


Figura 7 Exposición del servidor en el tráfico IP

Recomendaciones Técnicas

- Implementar CSP: Configurar la cabecera Content-Security-Policy para restringir la carga de contenido solo a orígenes de confianza, mitigando así ataques de inyección.
- Hardening de Red: Deshabilitar puertos y servicios innecesarios (especialmente el puerto 445 si no es requerido internamente) y asegurar que los motores SQL estén detrás de un firewall.

- Gestión de Errores Genéricos: Configurar la aplicación para que, ante cualquier fallo, el usuario solo reciba un mensaje genérico y un código de referencia, mientras que el error detallado se guarde en un log interno.
- Sanitización de Respuestas de API: Implementar filtros globales de excepciones que capturen errores internos y eviten que el stack trace llegue al cliente final.
- Ocultar Banner del Servidor: Configurar el servidor IIS para eliminar o anonimizar la cabecera Server de las respuestas HTTP, reduciendo el éxito de ataques dirigidos a versiones específicas

Checklist para Auditoría Continua (Andrómeda 2026)

Punto de Control	Estado de Cumplimiento
Auditar cabeceras de seguridad (CSP)	[✕] No Cumple
Evaluación de exposición de puertos (SQL/445)	[✕] No Cumple
Auditar mensajes de depuración expuestos	[✕] No Cumple
Mitigar la exposición de Stack Traces en APIs	[✕] No Cumple
Auditar la identificación del Sistema Operativo	[✕] No Cumple

A03:2025 – Fallas en Cadena de Suministro

Descripción Detallada

Este riesgo se centra en las fallas que ocurren cuando el software o los datos se utilizan sin verificar su integridad o su origen. En el desarrollo moderno, esto implica principalmente el uso de librerías y componentes de terceros que pueden contener vulnerabilidades conocidas o estar obsoletos, comprometiendo toda la cadena de suministro de la aplicación.

Ejemplos Específicos (General)

- Uso de librerías con CVEs: Implementar paquetes de JavaScript o NuGet que tienen fallos de seguridad documentados en bases de datos públicas.
- Dependencias sin mantenimiento: Utilizar componentes que ya no reciben parches de seguridad por parte de sus desarrolladores originales.
- Falta de verificación de firmas: Descargar y ejecutar scripts o plugins sin validar que el archivo no haya sido alterado durante la transferencia.

Malas Prácticas Comunes y Ejecución de Pruebas (Basado en el análisis de Andrómeda)

A. Evaluación de Librerías de Terceros (Análisis Dinámico)

Error común: Ignorar las alertas de seguridad de los componentes front-end que se cargan en el navegador.

Método: Durante el escaneo realizado con OWASP ZAP, la herramienta inspecciona las versiones de los scripts cargados en la aplicación.

Resultado del análisis: Se identificaron versiones específicas de Lodash y jQuery Validation que cuentan con registros activos en la base de datos de Vulnerabilidades y Exposiciones Comunes (CVE).

B. Análisis Automatizado de Componentes (SCA)

Herramienta**:** **Dependency-Check** (OWASP). Es un analizador que identifica las dependencias del proyecto y verifica si existen vulnerabilidades publicadas.

- Instalación y Uso:

Descargar el ejecutable desde el repositorio oficial (https://github.com/dependency-check/DependencyCheck/releases).

Obtener una NVD API Key (esencial para que la herramienta descargue la base de datos actualizada rápidamente) en el siguiente enlace: https://nvd.nist.gov/developers/request-an-api-key.

Ejecución: Se utiliza mediante la línea de comandos señalando el directorio del proyecto:

```
    ./bin/dependency-check.sh --nvdApiKey "TU_API_KEY" --project
"Andromeda" --scan ../andromeda/
```

Sintaxis: --nvdApiKey (autentica la conexión con la base de datos del NIST), --project (asigna un nombre al reporte), --scan (indica la ruta de la carpeta que contiene el código y las librerías).

Resultado del análisis: El escaneo genera un reporte en formato HTML (incluido en los entregables de la auditoría 2026) que detalla cada hallazgo. En el caso de Andrómeda, se detectó el complemento pdfmake en una versión obsoleta y vulnerable según los estándares actuales.

El archivo HTML generado por Dependency-Check es la evidencia técnica principal de esta sección y debe consultarse para conocer el listado completo de CVEs hallados durante el periodo enero-abril 2026.

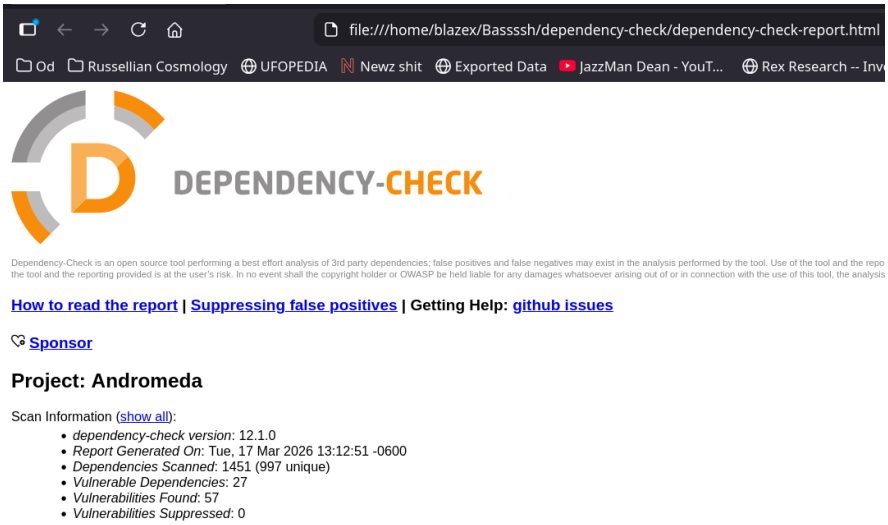


Figura 8 Reporte generado por Dependency-check

Recomendaciones Técnicas

- Actualización Constante: Establecer un ciclo mensual de actualización de dependencias de NuGet y NPM para asegurar que se utilicen versiones estables y parcheadas.
- Eliminación de Componentes Huérfanos: Remover del código cualquier librería o plugin que ya no sea esencial para la operación del sistema, reduciendo así la superficie de ataque.
- Uso de Herramientas de Escaneo en el CI/CD: Integrar herramientas como Dependency-Check de forma automática en el proceso de despliegue para bloquear versiones vulnerables antes de que lleguen a producción.

Checklist para Auditoría Continua (Andrómeda 2026)

Punto de Control	Estado de Cumplimiento
Auditoría de integridad y actualización de librerías/componentes de terceros	[✕] No Cumple

A04:2025 – Fallas Criptográficas

Descripción Detallada

Este punto se centra en la protección de datos en reposo y en tránsito. Las fallas ocurren cuando la información sensible no se cifra adecuadamente, se utilizan algoritmos obsoletos o se exponen llaves de seguridad, permitiendo que atacantes accedan a secretos del sistema o intercepten comunicaciones privadas.

Ejemplos Específicos (General)

- Uso de protocolos de transferencia de datos sin cifrado como HTTP o FTP.
- Almacenamiento de contraseñas en texto plano o con algoritmos de hash débiles (MD5, SHA1).
- Exposición de secretos de infraestructura (llaves de AWS, tokens de API) dentro del código fuente o archivos de configuración.

Malas Prácticas Comunes y Ejecución de Pruebas (Basado en el análisis de Andrómeda)

A. Análisis de Secretos en Código (SAST)

- Error común: Hardcodear credenciales, cadenas de conexión y llaves de servicios en la nube directamente en el código fuente.
- Método: Uso de Horusec, una herramienta de análisis estático diseñada para buscar vulnerabilidades y fugas de secretos en el repositorio.

- Instalación y Ejecución: Descargar el binario oficial (https://github.com/ZupIT/horusec) y ejecutar el siguiente comando dentro de la carpeta raíz del proyecto:

```
horusec start -p="./" -e="true" -o="json" -
O="./reporte_seguridad_14_ABRIL.json"
```

Sintaxis: -p (ruta del proyecto), -e (habilitar extensiones de análisis), -o (formato de salida), -O (nombre y ruta del archivo de reporte generado).

Resultado del análisis: La herramienta generó un reporte JSON que fue procesado en un dashboard personalizado para visualizar hallazgos por severidad (Critical, High, Warning). Para cargar el reporte en el dashboard es desde el botón “Cargar Reporte” y se carga el reporte json generado. Mediante este análisis se detectaron contraseñas en texto plano, llaves de API y tokens de AWS expuestos en diversas líneas de código.



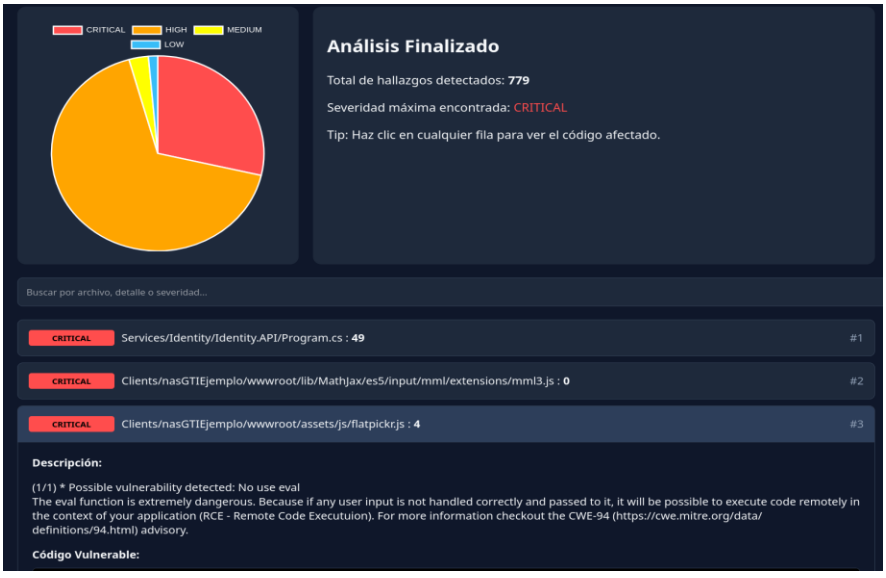


Figura 9 Gráfica con las vulnerabilidades identificadas en el código estático

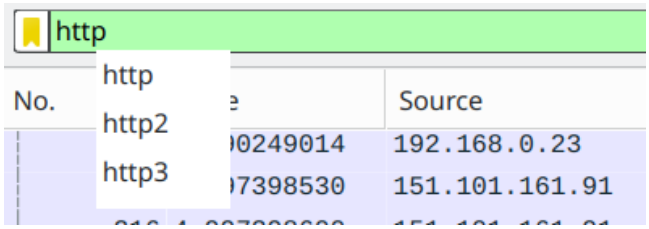
B. Auditoría de Cifrado en Tránsito

Error común: Permitir la comunicación mediante protocolos antiguos que son susceptibles a ataques de intercepción (Man-in-the-Middle).

Método: Análisis de protocolos con Wireshark.

Procedimiento:

1. Instalar Wireshark (https://www.wireshark.org/download.html) y seleccionar la interfaz de red activa.
2. Iniciar la captura de paquetes y realizar acciones sensibles en Andrómeda (login, consulta de modelos).
3. Filtrar el tráfico para verificar los protocolos de negociación de seguridad (por ejemplo filtrar por HTTPS o TLS).



Resultado del análisis: Se confirmó que la plataforma utiliza TLS 1.3, lo que asegura que toda la comunicación entre el cliente y el servidor esté cifrada robustamente, impidiendo la captura de credenciales en el tráfico de red.

34.107.243.93	192.168.0.23	TLSv1.3	684	Application Data, Application Data
34.107.243.93	192.168.0.23	TLSv1.3	97	Application Data
192.168.0.23	34.107.243.93	TLSv1.3	97	Application Data
192.168.0.23	34.107.243.93	TLSv1.3	1316	Client Hello (SNI=push.services.mozil
151.101.161.91	192.168.0.23	TLSv1.2	324	New Session Ticket, Change Cipher Spe
192.168.0.23	151.101.161.91	TLSv1.2	165	Application Data
192.168.0.23	151.101.161.91	TLSv1.2	277	Application Data
34.107.243.93	192.168.0.23	TLSv1.3	284	Server Hello, Change Cipher Spec, App
192.168.0.23	34.107.243.93	TLSv1.3	130	Change Cipher Spec, Application Data
192.168.0.23	34.107.243.93	TLSv1.3	692	Application Data

Figura 10 Captura de trafico con Wireshark

Recomendaciones Técnicas

- Uso de Gestores de Secretos: Migrar todas las credenciales y llaves de API fuera del código fuente hacia servicios como Azure Key Vault o variables de entorno cifradas.
- Eliminación de Texto Plano: Asegurar que ninguna contraseña o token sea legible en los archivos del repositorio; usar técnicas de inyección de secretos en tiempo de ejecución.

- Forzar HTTPS/TLS: Mantener la configuración de TLS 1.3 y deshabilitar versiones anteriores (TLS 1.0/1.1) para garantizar la integridad de los datos en tránsito.

Punto de Control	Estado de Cumplimiento
Análisis de secretos y contraseñas en código fuente	[✗] No Cumple
Auditoría de cifrado en tránsito (TLS 1.3)	[✓] Cumple

A05:2025 – Inyección

Descripción Detallada

Las fallas de inyección ocurren cuando se envían datos no confiables a un intérprete como parte de un comando o consulta. Los datos maliciosos pueden engañar al intérprete para que ejecute comandos involuntarios o acceda a datos sin la debida autorización.

Ejemplos Específicos (General)

- SQL Injection: Consultas dinámicas que concatenan la entrada del usuario directamente en la sentencia SQL.
- Cross-Site Scripting (XSS): Inyección de scripts maliciosos en páginas web que luego son ejecutados por el navegador de otros usuarios.
- HTML Injection (HTMLi): Inyección de etiquetas HTML para alterar la apariencia visual o el comportamiento de la interfaz.

Malas Prácticas Comunes y Ejecución de Pruebas (Basado en el análisis de Andrómeda)

A. Pruebas de Inyección de Bases de Datos

- Error común: Uso de concatenación de strings en lugar de consultas parametrizadas o un ORM configurado correctamente.
- Método: Escaneo automatizado con sqlmap. Para instalarlo en sistemas basados en Debian/Ubuntu usa **sudo apt install sqlmap**.

- Ejecución: Se ejecutó el comando apuntando al endpoint de autenticación con el siguiente desglose técnico:

```
sqlmap -u "https://vmgti06.ineel.mx/Home/Login" -p UserName --batch --data="UserName=admin&Password=admin&__RequestVerificationToken=TOKEN" --level=3 --risk=2 --random-agent --force-ssl
```

Sintaxis: -u (URL del objetivo), -p (parámetro específico a probar), --batch (no pedir entrada del usuario), --data (datos del formulario POST), --level/--risk (incrementan la profundidad de las pruebas), --force-ssl (obliga el uso de HTTPS).

Resultado del análisis: La plataforma demostró resiliencia, bloqueando y procesando correctamente los intentos de inyección SQL, validando la eficacia de la arquitectura .NET 8.

B. Pruebas de Inyección de Scripts (XSS/HTMLi)

- Error común: Falta de validación y sanitización de inputs en el servidor antes de renderizarlos en la vista.
- Método: Inyección manual de código en campos de texto y formularios de registro.
- Ejecución: Para XSS, se introducen comandos simples en la consola o inputs como: `<script>alert('XSS')</script>`. Para HTMLi, se introducen etiquetas de formato como `<u>Texto Subrayado</u>` o `<b>Negritas</b>` para observar si el navegador procesa el estilo.
- Resultado del análisis: Los bloques de código y etiquetas HTML ingresados fueron neutralizados y no se ejecutaron en el navegador, confirmando que la plataforma Andrómeda maneja correctamente la codificación de salida.

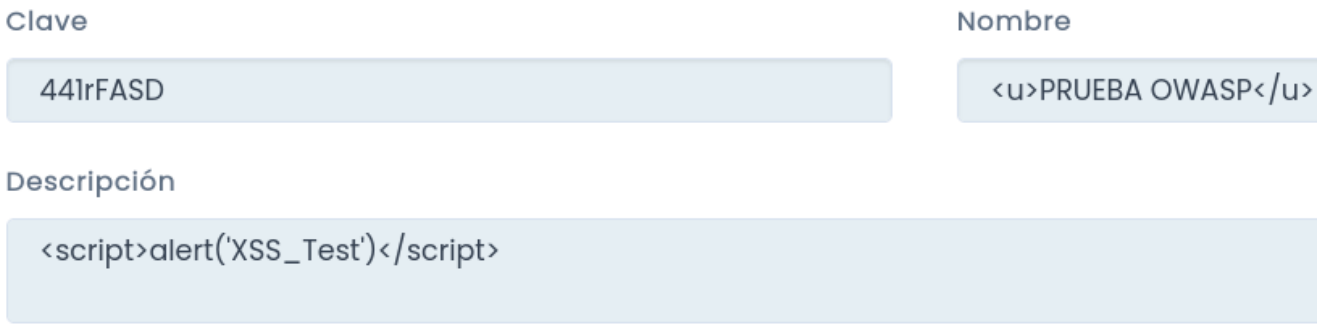


Figura 11 Inyeccion de codigo HTMLi

C. Análisis de Atributos de Seguridad en Cookies

- Error común: No configurar las banderas de seguridad en las cookies de sesión, permitiendo que scripts maliciosos accedan a ellas.
- Método: Análisis de tráfico de red mediante las herramientas de desarrollador del navegador (F12 -> Network) o Wireshark.
- Resultado del análisis: Se detectó la presencia del atributo HttpOnly, el cual asegura que las cookies solo sean procesadas por el servidor y no sean accesibles mediante JavaScript, mitigando ataques de robo de sesión.

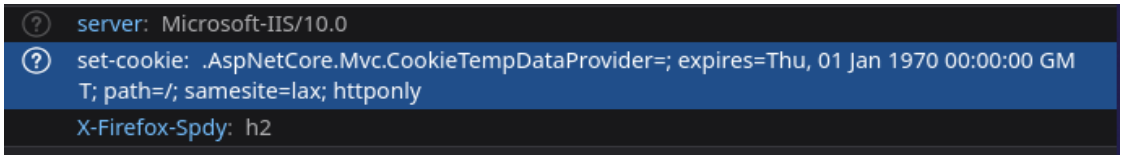


Figura 12 Atributo HttpOnly en las cookies

Recomendaciones Técnicas

- Uso de ORM: Mantener el uso de Entity Framework Core con consultas parametrizadas para evitar por completo la construcción manual de SQL.
- Validación Estricta: Implementar validación de entradas mediante listas blancas y asegurar que los frameworks de la interfaz (como Blazor o Razor) mantengan activa la codificación automática de salida.
- Configuración de Cookies: Asegurar que todas las cookies de sesión incluyan las banderas HttpOnly, Secure y SameSite=Strict para maximizar la protección del estado del usuario.

Checklist para Auditoría Continua (Andrómeda 2026)

Punto de Control	Estado de Cumplimiento
Pruebas de inyección de bases de datos (sqlmap)	[✓] Cumple
Pruebas de inyección de scripts (XSS/HTMLi)	[✓] Cumple
Análisis de atributos de seguridad en cookies (HttpOnly)	[✓] Cumple

A06:2025 – Diseño Inseguro

Descripción Detallada

El diseño inseguro se refiere a deficiencias en la arquitectura y el diseño del software que no pueden ser corregidas por una implementación perfecta, ya que el fallo reside en la concepción misma de los controles de seguridad. Se enfoca en la falta de perfiles de seguridad, patrones de diseño deficientes y la ausencia de mecanismos de protección ante amenazas lógicas.

Ejemplos Específicos (General)

- Falta de mecanismos de limitación de tasa (rate limiting) para prevenir ataques de fuerza bruta.
- Almacenamiento de información sensible en la memoria caché del navegador o en el historial sin políticas de limpieza.
- Ausencia de validación de estado en el flujo de navegación, permitiendo retroceder a páginas con datos cargados tras el cierre de sesión.

Malas Prácticas Comunes y Ejecución de Pruebas (Basado en el análisis de Andrómeda)

A. Persistencia de Datos en el Cliente

- Error común: Ausencia de políticas de control de caché (Cache-Control: no-store), lo que permite que información sensible permanezca accesible en el historial del navegador.
- Método: Pruebas manuales de navegación y gestión de sesiones.
- Procedimiento:

1. Iniciar sesión en la plataforma Andrómeda con un perfil administrativo.
2. Realizar consultas que carguen datos sensibles y modelos económicos en pantalla.
3. Ejecutar la acción de "Cerrar Sesión" para ser redirigido al formulario de Login.
4. Utilizar las flechas de navegación del navegador ("Atrás") para intentar regresar a las páginas previas.

- Resultado del análisis: Se observó que, a pesar de cerrar la sesión, el navegador permite visualizar nuevamente las páginas con información crítica previamente cargada. Los datos

permanecen en la caché y no se eliminan las sesiones de forma efectiva en el cliente, representando un riesgo de exposición si el equipo es compartido.

Recomendaciones Técnicas

- Implementar Filtros de Autenticación: Configurar un middleware global que valide la identidad del usuario en cada petición, asegurando que cualquier acceso de usuario anónimo sea redirigido estrictamente al Login.
- Políticas de Cache-Control: Configurar el servidor para enviar encabezados Cache-Control: no-store, no-cache, must-revalidate y Pragma: no-cache en todas las respuestas que contengan datos sensibles.
- Expiración Completa de Sesión: Asegurar que el proceso de logout elimine efectivamente todas las cookies de sesión, limpie el sessionStorage / localStorage del cliente y notifique a los proxies intermedios la invalidación del contenido.

Checklist para Auditoría Continua (Andrómeda 2026)

Punto de Control	**Estado de Cumplimiento**
Persistencia de datos en el cliente y gestión de caché al cerrar sesión	[✗] No Cumple

A07:2025 – Fallas en la Identificación y Autenticación

Descripción Detallada

Este riesgo ocurre cuando las funciones relacionadas con la autenticación y gestión de sesiones se implementan incorrectamente, permitiendo que atacantes comprometan contraseñas, tokens o claves de sesión, o que asuman identidades de otros usuarios de forma temporal o permanente.

Ejemplos Específicos (General)

- Permitir ataques de fuerza bruta al no limitar el número de intentos fallidos de inicio de sesión.
- Permitir el uso de contraseñas débiles o por defecto.
- No invalidar correctamente las sesiones tras un periodo de inactividad o permitir múltiples sesiones activas del mismo usuario sin control.

Malas Prácticas Comunes y Ejecución de Pruebas (Basado en el análisis de Andrómeda)

A. Gestión de Sesiones Simultáneas

- **Error común:** Permitir que una misma cuenta de usuario mantenga múltiples sesiones activas en diferentes dispositivos o navegadores simultáneamente, lo que aumenta la superficie de exposición en caso de robo de credenciales.
- **Método:** Pruebas manuales de acceso multi-dispositivo.
- **Procedimiento:** 1. Iniciar sesión en la plataforma Andrómeda desde un navegador específico (ej. Chrome). 2. Sin cerrar la primera sesión, intentar acceder con las mismas credenciales desde un navegador distinto (ej. Firefox) o desde una computadora diferente. 3. Realizar acciones de consulta en ambos navegadores al mismo tiempo.
- **Resultado del análisis:** El sistema permite la navegación activa y la ejecución de acciones en la misma cuenta desde tres navegadores distintos simultáneamente sin requerir re-autenticación o bloquear la sesión previa.

B. Resiliencia ante Ataques de Fuerza Bruta

- **Error común:** No contar con mecanismos de bloqueo de cuenta o retardos en las peticiones tras detectar múltiples intentos de acceso fallidos.
- **Método:** Pruebas manuales de intrusión (ataque de diccionario simulado).
- **Procedimiento:** Intentar ingresar a la plataforma repetidamente utilizando una cuenta válida pero con contraseñas erróneas de forma masiva.
- **Resultado del análisis:** La plataforma activó correctamente sus mecanismos de defensa integrados en el firewall, denegando el acceso total al servidor durante un periodo de 24 horas tras detectar los intentos fallidos masivos.

Recomendaciones Técnicas

- **Restricción de Sesiones Únicas:** Configurar el sistema para permitir solo una sesión activa por dispositivo, invalidando automáticamente la sesión anterior cuando se detecte un nuevo inicio de sesión.
- **Políticas de Re-autenticación:** Implementar mecanismos que soliciten nuevamente las credenciales para acciones críticas (como eliminación de modelos económicos) o tras periodos cortos de inactividad.
- **Refuerzo de Bloqueo de IP:** Mantener y optimizar las reglas del firewall para detectar patrones de ataque de fuerza bruta, pero permitiendo procesos de recuperación de cuenta seguros para usuarios legítimos.

Checklist para Auditoría Continua (Andrómeda 2026)

Punto de Control	Estado de Cumplimiento
Control y restricción de sesiones simultáneas	[✗] No Cumple
Resiliencia ante ataques de fuerza bruta (Firewall)	[✓] Cumple

A08:2025 – Fallas en la Integridad de Software y Datos

Descripción Detallada

Este riesgo se produce cuando el software y los datos se ven comprometidos debido a la falta de verificación de su integridad. En aplicaciones web, esto se manifiesta comúnmente en procesos de carga de archivos donde no se validan correctamente las extensiones, el contenido o la estructura de los datos importados, permitiendo la ejecución de código malicioso o la corrupción de la base de datos.

Ejemplos Específicos (General)

- Inclusión de archivos maliciosos mediante formularios de carga que no restringen tipos de archivos peligrosos (ej. .exe, .php, .js).

- Procesamiento de datos serializados o archivos de datos (XLSX, CSV) que contienen fórmulas maliciosas o scripts.

- Falta de validación de esquemas en la importación de datos, lo que permite la entrada de valores nulos o tipos de datos inconsistentes que rompen la lógica del negocio.

Malas Prácticas Comunes y Ejecución de Pruebas (Basado en el análisis de Andrómeda)

A. Validación de Tipos y Extensión de Archivos

- **Error común:** Confiar únicamente en la validación del lado del cliente (frontend) para restringir extensiones de archivos, permitiendo que un atacante salte esta restricción mediante interceptores de tráfico.
- **Método:** Pruebas manuales de carga de archivos con extensiones no autorizadas.
- **Procedimiento:** 1. Acceder a los módulos de carga de información de Andrómeda (diseñados para archivos .xlsx). 2. Intentar cargar archivos con extensiones prohibidas como .pdf, .exe o .js.
- **Resultado del análisis:** Se confirmó que la plataforma cuenta con reglas de validación robustas en el servidor que restringen los módulos de carga, permitiendo exclusivamente las extensiones requeridas para el análisis de datos.



Figura 13 Validación de los tipos de archivos

B. Validación de Integridad y Estructura de Datos

- **Error común:** Procesar archivos de hojas de cálculo sin validar el contenido de las celdas, permitiendo la entrada de basura técnica o inyecciones de fórmulas.
- **Método:** Pruebas manuales de carga con datos corruptos y malformados.
- **Procedimiento:** 1. Preparar un archivo de Excel con celdas que contengan valores nulos, errores de división (#DIV/0!), cadenas de texto en campos numéricos y comandos incompletos o fórmulas inusuales (=sum, cmd()). 2. Cargar dicho archivo en el sistema para observar el comportamiento de procesamiento.
- **Resultado del análisis:** La plataforma demostró una alta resiliencia al denegar la carga de tablas con errores estructurales. El sistema identifica los datos incorrectos y asegura que solo la información íntegra y válida sea procesada por los modelos de Andrómeda.

Recomendaciones Técnicas

- **Validación de Firma de Archivos:** Además de la extensión, implementar la verificación de "Magic Numbers" (firmas de archivo) en el servidor para asegurar que un archivo .xlsx sea realmente un archivo de Excel y no un ejecutable renombrado.
- **Sanitización de Importaciones:** Utilizar librerías de procesamiento de datos que deshabiliten automáticamente la ejecución de macros o fórmulas externas al importar archivos de terceros.
- **Esquemas de Datos Estrictos:** Mantener validaciones de tipo de dato fuertes en los modelos de C# (.NET 8) para asegurar que cualquier valor nulo o inconsistente sea rechazado antes de llegar a la capa de persistencia.

Checklist para Auditoría Continua (Andrómeda 2026)

Punto de Control	Estado de Cumplimiento
Validación de tipos y extensión de archivos en servidor	[✓] Cumple
Validación de integridad y estructura de datos importados	[✓] Cumple

A09:2025 – Fallas en el Registro y Monitoreo Control

Descripción Detallada

Este riesgo se refiere a la incapacidad de una plataforma para detectar, escalar y responder a brechas de seguridad en tiempo real. Sin un registro adecuado y un monitoreo activo, los atacantes pueden permanecer dentro de la red durante periodos prolongados sin ser detectados, realizando movimientos laterales o exfiltración de datos.

Ejemplos Específicos (General)

- Eventos auditables, como inicios de sesión fallidos o transacciones de alto valor, que no quedan registrados en ninguna bitácora.
- Logs que solo se almacenan localmente en el servidor sin una centralización que permita su análisis tras un compromiso del sistema.
- Alertas de seguridad que no se disparan ante patrones de tráfico sospechoso o ataques de fuerza bruta conocidos.

Malas Prácticas Comunes y Ejecución de Pruebas (Basado en el análisis de Andrómeda)

A. Evaluación de Centralización de Logs en Aplicación

- **Error común:** Generar mensajes de error que el usuario ve, pero que no quedan guardados en una base de datos o archivo de auditoría para el equipo de seguridad.
- **Método:** Análisis administrativo y de servidor mediante pruebas de error forzadas.
- **Procedimiento:** Realizar acciones que generen excepciones en la plataforma y verificar si estas quedan registradas con detalles como la marca de tiempo, el usuario y el tipo de error.
- **Resultado del análisis:** La plataforma Andrómeda cumple satisfactoriamente en este nivel, contando con un registro de logs y alertas funcional para todos sus procesos internos de aplicación.

B. Gestión de Alertas y Logs a Nivel de Red

- **Error común:** Ignorar la visibilidad del tráfico de red y la infraestructura, dejando "puntos ciegos" donde un atacante puede operar a nivel de sistema operativo o puerto sin ser visto.
- **Método:** Análisis de persistencia y gestión de logs ante tráfico sospechoso (Rol de atacante).
- **Procedimiento:** Intentar verificar si los escaneos de Nmap o ataques de diccionario realizados previamente dispararon alertas automáticas visibles para el administrador de infraestructura.
- **Resultado del análisis:** Este punto se encuentra en **Estado Pendiente**. Debido a limitaciones en los permisos de acceso administrativo a la infraestructura profunda del servidor, no fue posible confirmar si el tráfico sospechoso generado durante la auditoría fue debidamente alertado.

Recomendaciones Técnicas

- **Implementar un SIEM:** Centralizar los logs de la aplicación y del servidor en una herramienta de Gestión de Eventos e Información de Seguridad (SIEM) para correlacionar eventos de red con acciones del usuario.
- **Alertas de Tráfico Sospechoso:** Configurar umbrales de alerta automatizados que notifiquen al equipo de seguridad ante escaneos de puertos masivos o intentos de inyección detectados por el firewall.
- **Integridad de Logs:** Asegurar que los archivos de log tengan permisos de "solo adición" y que se realice una copia de seguridad externa para evitar que un atacante borre sus huellas tras un compromiso.

Checklist para Auditoría Continua (Andrómeda 2026)

Punto de Control	Estado de Cumplimiento
Registro de logs y errores a nivel de plataforma	[✓] Cumple
Evaluación de logs y alertas de red ante tráfico sospechoso	[❗] PENDIENTE

A10:2025 – Mal Manejo de Condiciones Excepcionales

Descripción Detallada

Este riesgo se refiere a la incapacidad de una aplicación para manejar adecuadamente estados inusuales o entradas corruptas. Una gestión deficiente de estas condiciones puede provocar desde la denegación de servicio (DoS) hasta la exposición de errores técnicos que facilitan el reconocimiento del sistema por parte de un atacante.

Ejemplos Específicos (General)

- Caída del servidor al recibir tipos de datos no esperados (ej. texto en campos de fecha).
- Procesamiento de solicitudes malformadas que consumen recursos excesivos de CPU o memoria.
- Falta de límites en el tamaño o frecuencia de las peticiones, permitiendo condiciones de carrera o saturación del sistema.

Malas Prácticas Comunes y Ejecución de Pruebas (Basado en el análisis de Andrómeda)

A. Pruebas de Fuzzing (Entradas Inesperadas)

- Error común: Confiar en que el usuario solo enviará datos a través de la interfaz visual, ignorando que los paquetes pueden ser interceptados y modificados directamente.
- Herramienta: Burp Suite.
- Procedimiento:
- Instalación:

1. Descargar e instalar Burp Suite Community Edition (https://portswigger.net/burp/communitydownload).
2. Configuración de Proxy: Configurar el navegador para dirigir el tráfico a través del puerto 8080 y activar el modo "Intercept is ON" en Burp Suite.

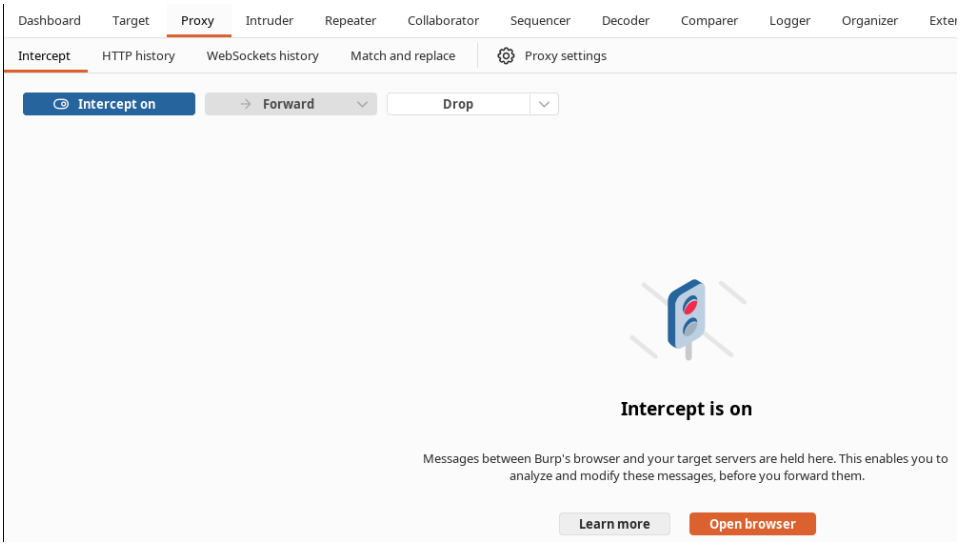


Figura 14 Modo intercept en Burp Suite

3. Intercepción de Paquetes: Iniciar una consulta legítima en Andrómeda y capturar el paquete antes de que llegue al servidor.
4. Alteración de Datos: Modificar el contenido del paquete sustituyendo números por cadenas de texto largas, símbolos especiales o fechas inexistentes (ej. "2026-13-40").

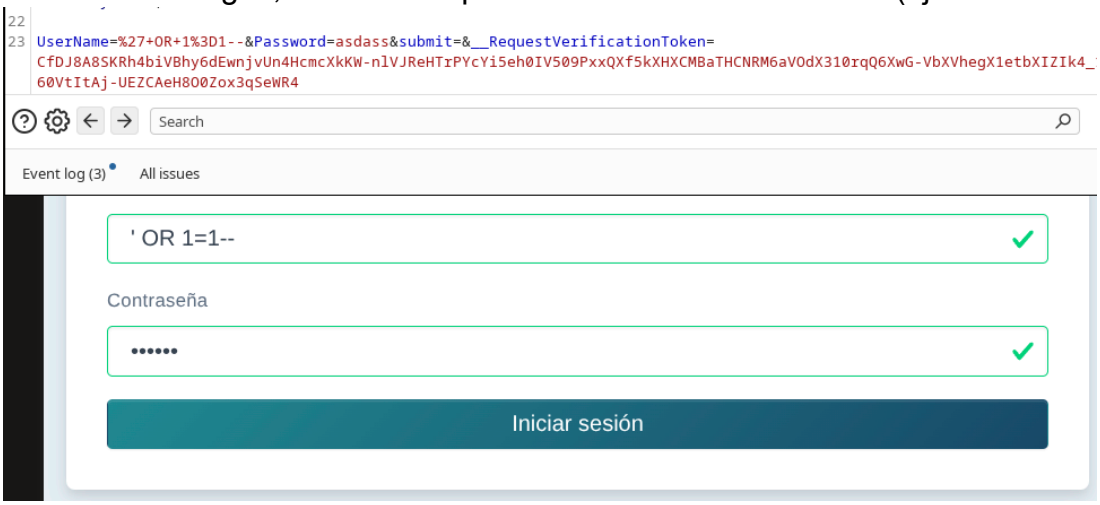


Figura 15 Ejemplo de inyección SQL con Burp Suite

5. Forward: Enviar el paquete alterado al servidor y observar la respuesta del sistema.

Resultado del análisis: La plataforma procesó adecuadamente los datos malformados, mostrando mensajes genéricos en la interfaz en lugar de errores técnicos y registrando el evento en los logs internos para su atención. El sistema no permitió que las peticiones alteradas corrompieran la lógica del proceso.

Recomendaciones Técnicas

- Validación de Tipos en el Lado del Servidor: No depender de las restricciones del navegador; implementar validaciones estrictas en los controladores de .NET para asegurar que cada parámetro cumpla con el formato esperado.
- Implementar Rate Limiting: Configurar límites de peticiones por IP para evitar que pruebas de fuzzing masivas o ataques de estrés saturen los recursos del servidor IIS 10.0.
- Manejo Global de Excepciones: Asegurar que cualquier condición excepcional no controlada sea capturada por un filtro global que retorne un código HTTP 400 (Bad Request) limpio, sin revelar detalles del stack trace.

Checklist para Auditoría Continua (Andrómeda 2026)

Punto de Control	Estado de Cumplimiento
Resiliencia ante entradas inesperadas y pruebas de fuzzing	[✓] Cumple

Conclusión de la Auditoría

Es fundamental comprender que el estándar OWASP Top 10 utilizado en esta auditoría no es una regla estática, sino una metodología adaptable. Representa los diez riesgos de seguridad más críticos en aplicaciones web, pero debe ser ajustado según las tecnologías específicas, como la arquitectura de microservicios o el uso de frameworks como .NET 8.

OWASP funciona como una enciclopedia global de ciberseguridad, ofreciendo recursos que van más allá de una simple lista. Para profundizar en la seguridad de los sistemas, es recomendable investigar y basarse en los puntos conocidos como CWE (Common Weakness Enumeration), que proporcionan un lenguaje común para describir las debilidades de seguridad en el software y sirven como base para identificar, mitigar y prevenir vulnerabilidades desde la fase de diseño.